Understanding Laravel 8

APPLICATION DEVELOPMENT

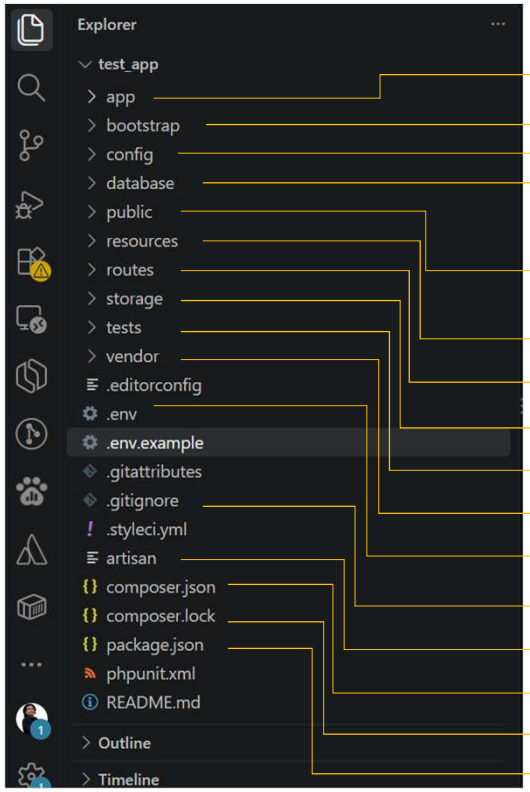
Pre-requisites:

- You already created a Laravel 8 project
- Xampp or Laragon is already working

How to start a Laravel dev server?

- Open your VS Code
- Browse on your navigation at the top and click **File > Open folder**
- Select your Laravel project
- Open your VS Code terminal and run this command:
  php artisan serve
- Open browser like Chrome, Brave, Edge etc… and browse:
  http://127.0.0.1:8000

Laravel folder structure



app - Your application's PHP code. This is where you'll commonly work with controllers and models.

bootstrap - Files Laravel uses when starting the application. You rarely need to edit this as a beginner.

config - Settings for how the application works, such as database and mail configuration.

database - Database migrations, seeders, and factories. A migration describes changes to database tables.

public - The entry point for web requests. Publicly accessible files, such as built CSS and JavaScript, go here.

resources - Source files for your pages and interface, such as Blade views, CSS, and JavaScript.

routes - Defines which code runs when someone visits a URL or calls an API.

storage - Files the application generates, including logs, cache, and uploaded files.

tests - Code that checks whether your application behaves as expected.

vendor - Laravel and other PHP packages installed by Composer. Composer manages this folder for you.

.env - Your local settings, such as database credentials and app URL. Keep this file private.

.gitignore - Tells Git which files and folders to leave out, such as .env and vendor.

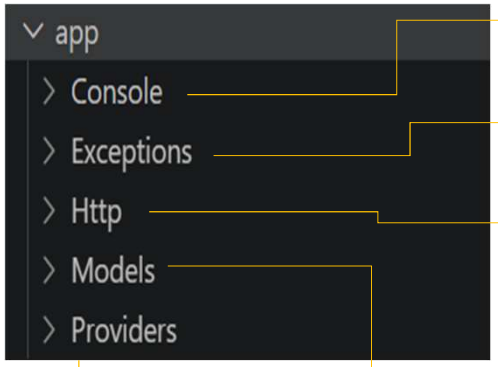
artisan - Laravel's command-line tool. For example, php artisan serve starts a local development server.

composer.json - Lists the PHP packages the project needs.

composer.lock - Records the exact installed versions of those PHP packages.

package.json - Lists JavaScript tools and packages used by the project.

The app Directory



Console - Code for tasks you run by typing a command in the terminal.

Exceptions - Code that helps Laravel handle errors.

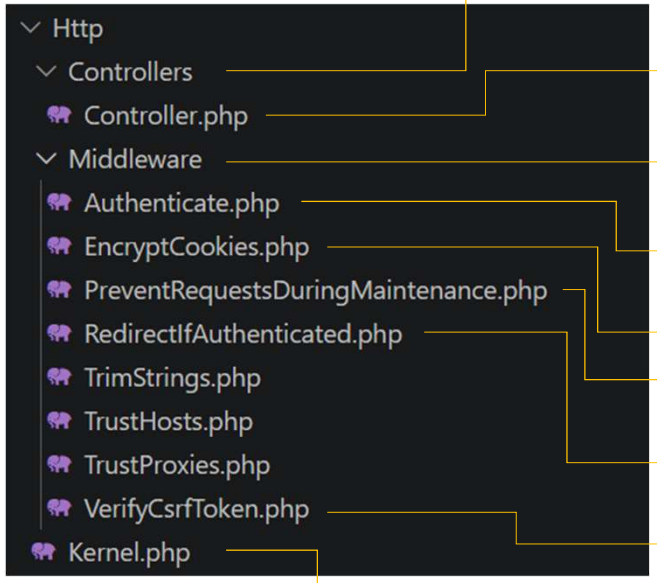
Http - Code that handles requests from a browser or an API. This is where you'll usually find **controllers**.

Models - Code that represents and works with your application's data, often stored in database tables.

Providers - Code that helps set up services when Laravel starts. Beginners usually don't need to change it yet.

The app/http Directory

The app/Http folder contains code that helps Laravel handle a **request**—for example, when someone opens a page, submits a form, or calls an API.



Controllers - Contains code that decides what to do with a request. A controller might get student records and show them on a page.

Controller.php - A base file that other controllers can build on. Students usually create their own controllers alongside it.

Middleware - Contains checks or actions that run as a request passes through Laravel. Think of middleware as a checkpoint before the controller runs.

Authenticate.php - Checks whether the visitor is logged in before allowing access to a protected page.

EncryptCookies.php - Protects cookie values so they cannot be easily read or changed.

PreventRequestsDuringMaintenance.php - Stops normal visits while the application is in maintenance mode.

RedirectIfAuthenticated.php - Sends someone who is already logged in away from pages such as the login page.

VerifyCsrfToken.php - Checks a security token when a form is submitted, helping prevent unwanted submissions from another website.

Kernel.php - Helps organize which middleware Laravel uses. This screenshot shows a Laravel structure that includes an HTTP kernel.

Laravel Model

A **model** is a PHP class that lets your application work with data in a database table.

For example, a **Student** model can read, add, update, and delete records in a students table.

“The model is how Laravel talks to a table.”

How to create one:

Open a terminal in your Laravel project and run:

php artisan make:model Student

Laravel creates app/Models/Student.php. A basic model looks like this:

Student.php

app > Models > Student.php > ...

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Student extends Model
{
    use HasFactory;
}

```

By convention, Laravel connects the Student model to a table named students. The command **creates the model file, but does not create the database table**. To create a model together with a migration for its table, use:

php artisan make:model Student –m

A **migration** is a file where you describe the table's columns, such as name and email.

Laravel Migration

A **migration** is a PHP file that describes the structure of a database table. Think of it as a **plan for building or changing a table**.

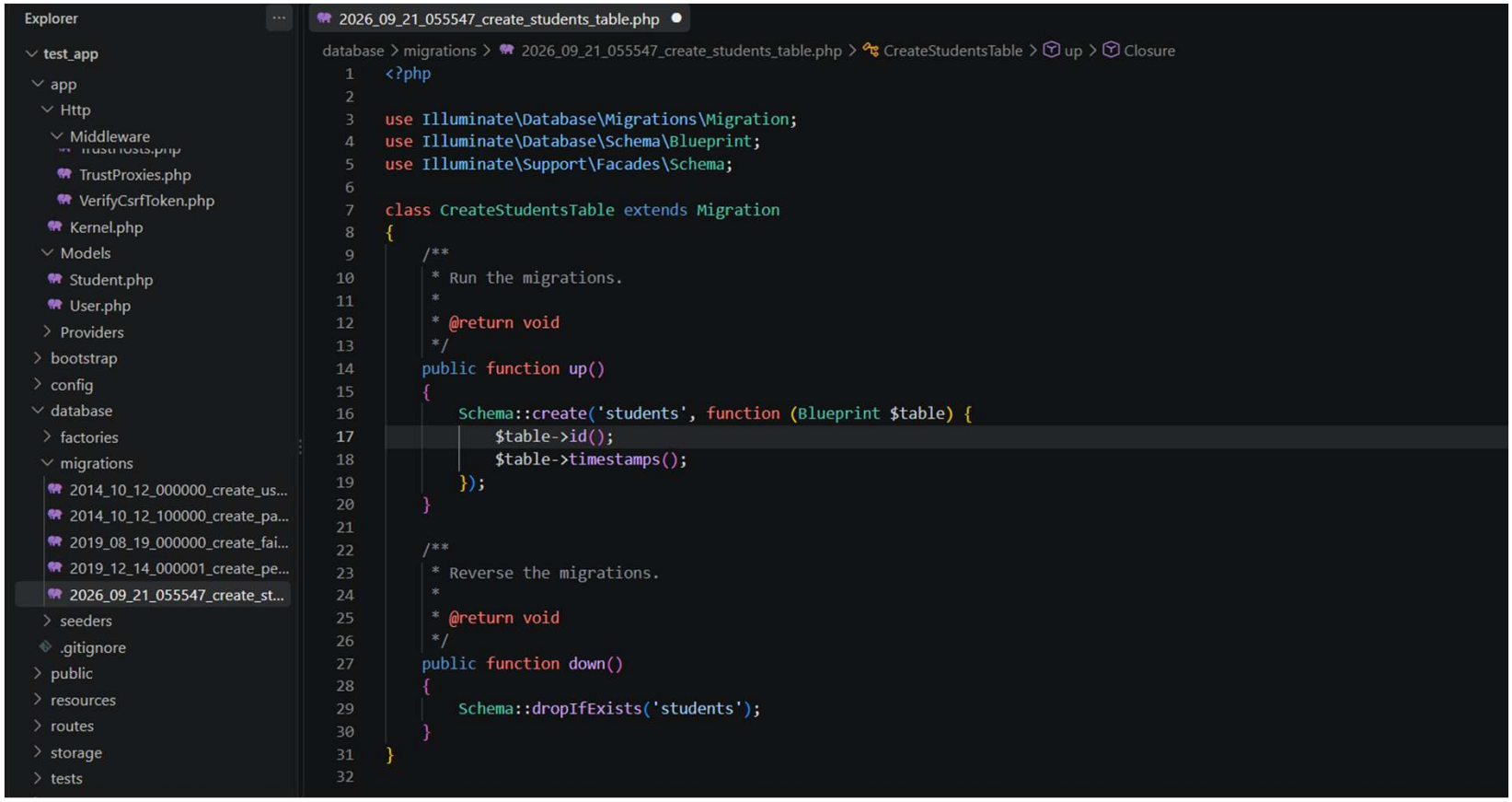
For example, a migration for students can specify columns for name, email, and created_at.

Other way to create a migration file:

If you already created the Student model, run:

php artisan make:migration create_students_table

Laravel will add a new file in database/migrations.


Explorer
test_app
app
Http
Middleware
TrustProxies.php
VerifyCsrfToken.php
Kernel.php
Models
Student.php
User.php
Providers
bootstrap
config
database
factories
migrations
2014_10_12_000000_create_us...
2014_10_12_100000_create_pa...
2019_08_19_000000_create_fai...
2019_12_14_000001_create_pe...
2026_09_21_055547_create_st...
seeders
.gitignore
public
resources
routes
storage
tests
2026_09_21_055547_create_students_table.php
database > migrations > 2026_09_21_055547_create_students_table.php > CreateStudentsTable > up > Closure
<?php
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;
class CreateStudentsTable extends Migration
{
/**
* Run the migrations.
*
* @return void
*/
public function up()
{
Schema::create('students', function (Blueprint $table) {
$table->id();
$table->timestamps();
});
}
/**
* Reverse the migrations.
*
* @return void
*/
public function down()
{
Schema::dropIfExists('students');
}
}

Modify the Student model

Add columns your table needs inside the up() method:

```php
public function up()
{
    Schema::create('students', function (Blueprint $table) {
        $table->id();
        $table->string('name');
        $table->string('student_id');
        $table->string('address')->nullable();
        $table->string('contact_no')->nullable();
        $table->timestamps();
    });
}
```

In your terminal, run this command to migrate: **php artisan migrate**

This applies migrations that have not yet run and creates the students table. The down() method describes how to undo the change, usually by dropping the table.

Laravel Controller

A **controller** is a PHP class that handles a request and decides what the application should do.

For example, when someone visits a student list page, a controller can get the student records and pass them to a view for display.

Think of it as: **Route → Controller → Model → View**

How to create a controller

In your Laravel project's terminal, run:

php artisan make:controller StudentController

Laravel creates **app/Http/Controllers/StudentController.php**. Add an index() method to show the student list:

Modify StudentController.php

2026_09_21_055547_create_students_table.php | Student.php | StudentController.php

app > Http > Controllers > StudentController.php > ...

```php
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use App\Models\Student;

class StudentController extends Controller
{
    public function index()
    {
        $students = Student::all();

        return view('students.index', compact('students'));
    }
}

```

Here, Student::all() gets all student records. view('students.index', ...) sends them to resources/views/students/index.blade.php for display.

Laravel Routes

A **route** tells Laravel what to do when someone visits a particular URL. It connects a web address to code that handles the request.

For example: **“When someone visits /students, run the index() method of StudentController.”**

How to create a route?

Open **routes/web.php** and add:

routes > web.php > ...

```php
<?php

use App\Http\Controllers\StudentController;
use Illuminate\Support\Facades\Route;

/*
|--------------------------------------------------------------------------
| Web Routes
|--------------------------------------------------------------------------
|
| Here is where you can register web routes for your application. These
| routes are loaded by the RouteServiceProvider within a group which
| contains the "web" middleware group. Now create something great!
|
*/

Route::get('/students', [StudentController::class, 'index']);

Route::get('/', function () {
    return view('welcome');
});

```

Now, when a user visits **/students**, Laravel calls the controller's index() method. The controller gets the students and returns the student list page.

Understanding the code:

```
Route::get('/students', [StudentController::class, 'index']);
```

Part	Meaning
Route::get()	Handles a request to **view** a page.
'/students'	The URL the user visits.
StudentController::class	The controller Laravel should use.
'index'	The method Laravel should run inside that controller.

The flow so far is: **URL (/students) → route → controller → model → view**.

Important commands for routes:

- php artisan route:list // See all registered routes.
- php artisan route:clear // Clear the route cache if Laravel is using an older cached list of routes.

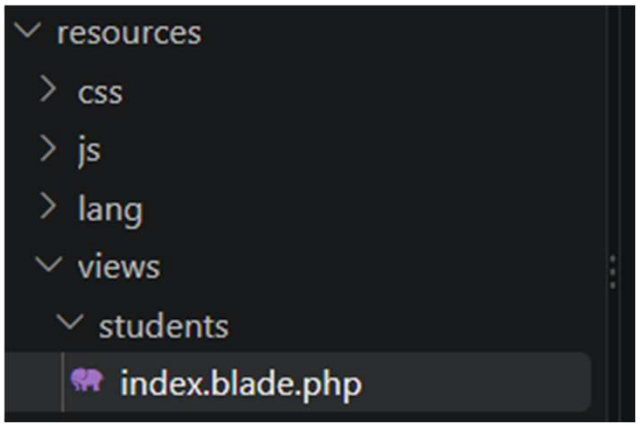
Laravel Views

- A **view** is the file that creates the page a user sees in the browser. Laravel commonly uses **Blade** for views. Blade lets you write HTML and display data sent by a controller.

How to create a view file?

Inside resources/views, create a students folder. Then create:

resources/views/students/index.blade.php



The .blade.php ending tells Laravel that this is a Blade view.

Add a page content inside your index.blade.php

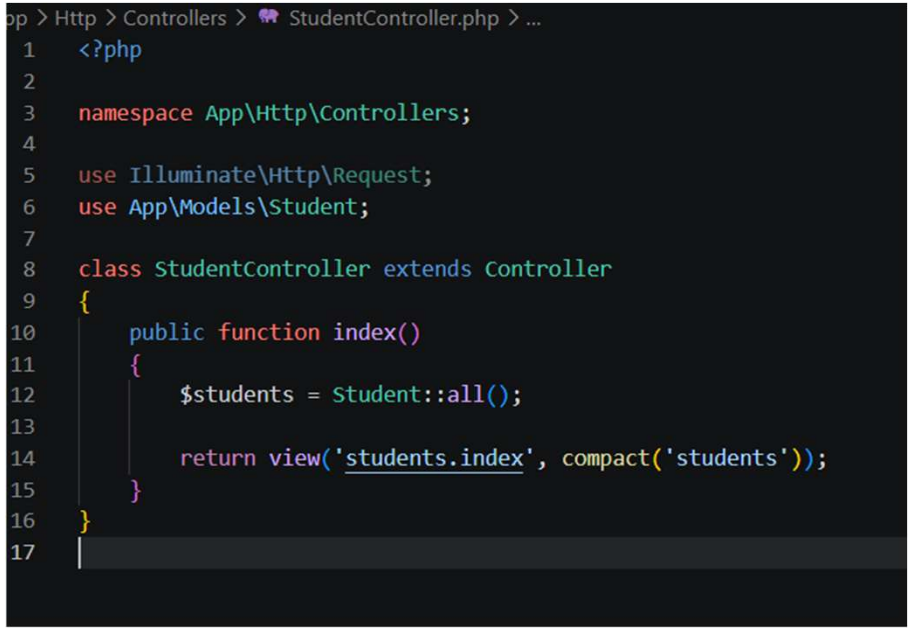
index.blade.php

resources > views > students > index.blade.php > html

```
1  <!DOCTYPE html>
2  <html>
3
4  <head>
5      <title>Students</title>
6  </head>
7
8  <body>
9      <h1>Student List</h1>
10
11     <ul>
12         @foreach ($students as $student)
13             <li>{{ $student->name }}</li>
14         @endforeach
15     </ul>
16 </body>
17
18 </html>
19
```

@foreach repeats the <li> for every student. {{ $student->name }} displays each student's name.

How it works?



In StudentController, the index() method gets the records and passes them to the view:

students.index points to resources/views/students/index.blade.php. Once the /students route is set up, visiting that URL will show the list.

The **model** gets the data, the **controller** prepares it, and the **view** displays it.

Activity 2

- Create a Laravel page that displays student records. Follow the code snippets from the previous slides.
- Create the Student model and migration using:
  - **php artisan make:model Student -m**
- Add the following columns to the migration: student_id, name, address, and contact_no. Run the migration to create the table.
- Create the view. Inside resources/views, create a folder named students and add an index.blade.php file.
- Create the Student controller using:
  - **php artisan make:controller StudentController**
- Add an index() method that retrieves student records and passes them to the **students.index** view.
- Add the route. Update **routes/web.php** with a **/students** route that points to the index() method of **StudentController**.
- Display the student records. In **index.blade.php**, display a table containing each student's ID, name, address, and contact number. Add sample records to your database to test the page.
- Submit your work. Push your project to a new public GitHub repository, then submit the repository link through the Google Form: https://forms.gle/o9pbm7bYUgDTJkF3A

END